Smart University
Resource Allocation System

Iteration 43

Student-Initiated University Room Allocation

Najib Attar | UC-STU-02 | October 2026

1. Iteration Goal

Deliver one complete student room-request workflow: students state their needs, C++ identifies suitable rooms, and the selected room's owning faculty reviews the exact assignment. Preserve appointments, PostgreSQL authority, existing weekly allocation and all five academic allocation strategies.

2. Changes from Iteration 42

Iteration 42 baseline	Iteration 43 addition
28 application tables; authenticated appointments	Nine additive room/policy tables; 37 application tables total
Professor approval of room-independent appointments	Faculty approval of one exclusive student room reservation
Allocation JSON v1 and appointment operation v1	Separate studentRoom operation v1 via --rooms-json
Personal timetable and student Home	Room requests, optional professor attendance and own room bookings

3. Scenario

A student describes a study session for five attendees, date/time, duration and required facilities. The system proposes a ranked room, including eligible rooms in another faculty. Submission is pending and holds nothing. The currently designated administrator of the selected room's faculty rechecks the exact room and revision before confirmation. A competing reservation makes the request require revision, not an unapproved substitution. Optional professors respond independently. Cancellation releases time and retains history.

4. Modules and Their Tasks

Component	Responsibility
RoomRequestService / room blueprint	Authenticated lifecycle, faculty authorization, revisions, invitations, configuration and thin forms
RoomSelectionService / RoomRequestJsonProtocol	Eligibility, conflict checks and deterministic single-room ranking in C++
room_calendar / EngineGateway	Explicit dated projections, verified executable, JSON transport and response checks
PostgreSQL / personal schedule services	Atomic reservations/history and private room/appointment timetable integration

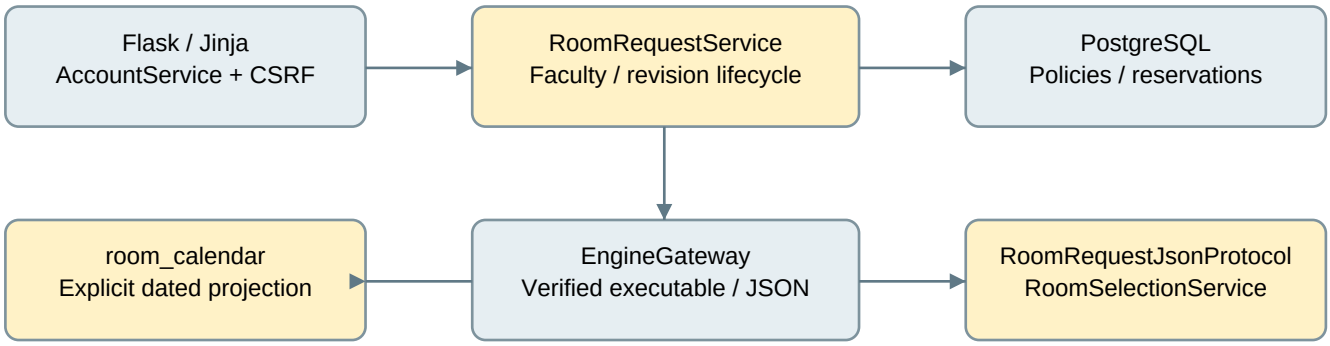
5. Database and Relationships

Alembic revisions 0043a and 0043b add nine tables without inventing owners or academic dates. Existing 28 tables remain. Resources are not student-bookable until explicitly configured. Alembic's own version table is infrastructure, not an application entity.

New table	Purpose / principal references
faculty_room_approver	One designated administrative login for each owning Faculty organization unit
room_policy	Resource identity, owning faculty, enabled/categories and access policy
room_access	Composite resource/user whitelist for restricted rooms
room_closure	Date-specific resource unavailability
student_room_request	Organizer, exact selected resource, faculty/approver, interval, needs, revision/status
room_invitation	Optional professor and current revision response
room_reservation	Exact confirmed exclusive resource/time; one per confirmed room request
room_organizer_commitment	Exact confirmed organizing-student/time; cross-appointment protection
room_event	Private lifecycle history, actor, request revision and timestamp

All references use existing university identities, resources and organization units. Primary/foreign keys, bounded checks, deferred aggregate triggers and GiST interval exclusions protect committed shape. Confirmed requests require approval of the current revision and exact room/organizer reservations; other states cannot hold reservations. Adjacent intervals are allowed. Shared academic exam allocation is not forbidden by a blanket SQL room exclusion.

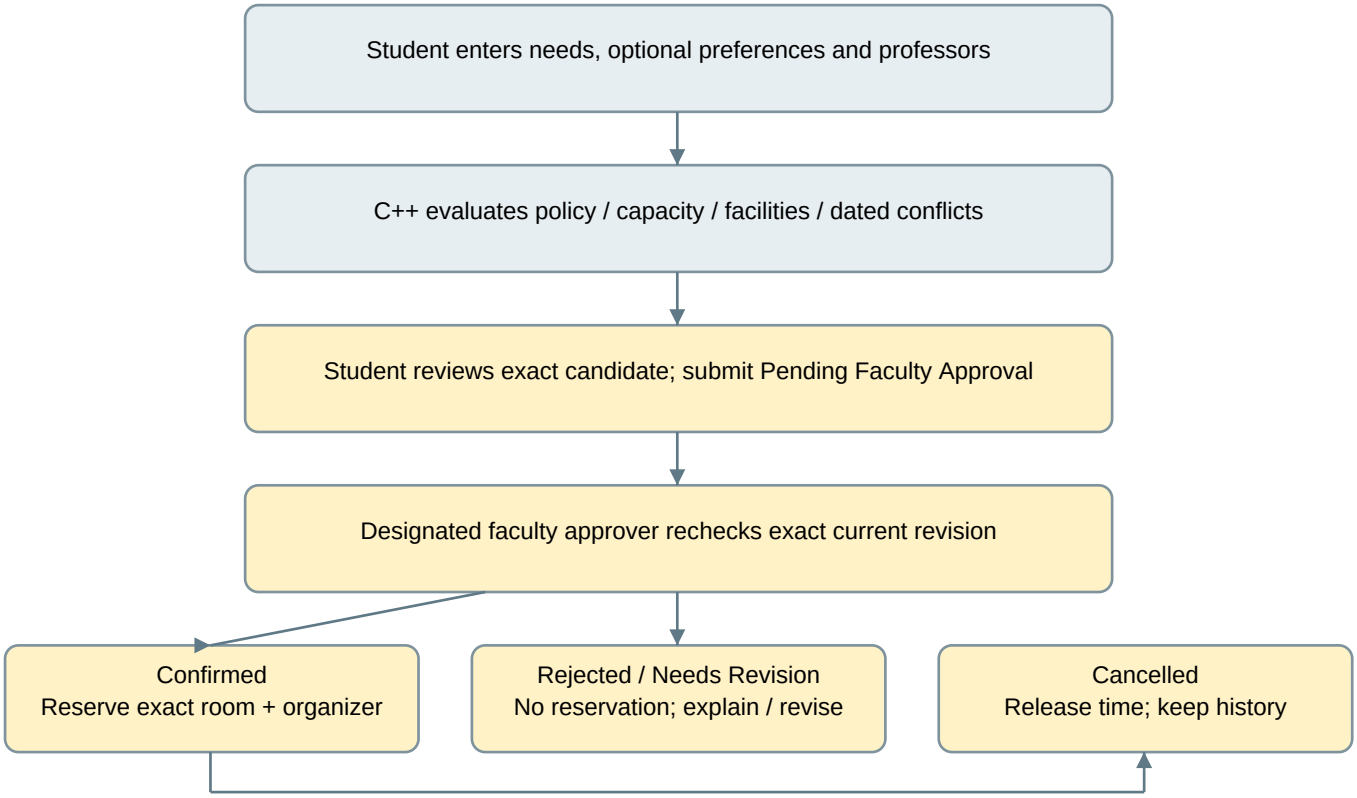
6. Updated Component Diagram



New room-specific components are highlighted. Existing engine strategies remain separate.

The complete physical/high-level ER sources, zoomable SVGs and schema catalog are maintained under docs/architecture/. The student-room component source/render accompanies the design. The diagram above summarizes boundaries rather than replacing the full ER.

7. Updated Workflow



8. Prototype Implementation and Safety

Selection. Best-fit ranks unmatched preferences, spare seats, unnecessary facilities, then room ID. Greedy preserves source ordering within preference ties. The system setting is not exposed as an academic strategy choice to students. No exact-time candidate triggers up to five feasible alternative-time suggestions. Every chosen alternative is freshly evaluated.

Calendar. Configurable Europe/Istanbul and half-open intervals coexist with the unchanged weekly TimeSlot. Academic requests require explicit period/planning-week bindings. Unresolved applicability blocks the affected room or mandatory organizer; dates are never inferred. Closures, confirmed appointments and student reservations are included where relevant.

Transactions. C++ runs outside write transactions. Final approval locks existing application state, checks calendar/request revisions, and commits reservations, decision and history together. Concurrent confirmations cannot double-book. Later academic publication and binding changes use the same conflict gate, preserving the last valid published run on failure.

Authorization and privacy. Organizer, assigned approver and invitees have query-scoped detail access. Only the currently designated room-faculty approver decides. CSRF and signed sessions remain active. Other permitted calendar readers see free/busy, not private purpose or participants. Optional RSVP is checked against the professor's calendar but does not exclusively reserve it or veto faculty approval.

Patterns. Existing Factory-style space creation, polymorphism and EngineGateway indirection are reused. A focused application service separates lifecycle/transactions from routes and C++ decisions. No new framework or pattern was introduced merely for its name.

9. Verification

Executed check	Result / scope
Complete Windows C++ runner	160 passed, 0 failed; includes 148 previous and 12 new room tests
Isolated standalone CSV CLI smokes	Two smoke scenarios passed; existing generated CSV files unchanged
Complete Python / real PostgreSQL suite	246 executed: 245 passed, 0 failed, 1 POSIX-only skip on Windows
Focused room / database completion suite	38 passed; includes thread/process confirmation competition and revision checks
Fictional local demo regression	4 passed; 106 users, 94 accounts, 20 room policies, two faculty approvers
Browser acceptance	Student candidate/submission, faculty confirmation, professor RSVP, own timetable and cancellation verified
Syntax / schema documentation	Python compile, JavaScript syntax and all-37-table source/catalog check passed

Commands: powershell -ExecutionPolicy Bypass -File tests/run_tests.ps1; python -m unittest discover -s tests -p test_*.py. PostgreSQL verification used uniquely owned disposable schemas/databases. The suite covers upgrades/drift, rollback/recovery, all-five-strategy parity, existing appointment/JSON behavior, room privacy and concurrent confirmation. Failure-injection errors are expected assertions, not successful operational runs. Remote CI is separately reported in the pull request; local evidence is not a claim that an unexecuted Linux run passed.

10. Demonstration and Remaining Boundaries

The local launcher creates a separate fictional student-rooms profile with explicit January 7-31, 2030 calendar data and 79 allocations produced by the real engine. Sign in as student1, choose Find a room, use a fictional January date and submit. The detail page names admin/admin2 as assigned faculty approver. Confirm there, return to the student's January timetable, and cancel to demonstrate release. Passwords/settings remain private and are never committed.

Notifications are in-app inbox/status/history, not email or push. Club affiliation is declared rather than verified membership. Existing resources require explicit trusted policy setup. Optional professor attendance is not a hard commitment. Full-batch academic projections, list/detail queries and retained history suit the demonstration scale; deployment hardening, pagination and retention remain separate concerns. No production migration, deployment or Iteration 43 merge is implied.

11. Summary

UC-STU-02 connects a student-facing room selection workflow to authoritative C++ feasibility and PostgreSQL-safe faculty approval. It preserves the existing allocation strategies, appointment service and both previous JSON operations while adding explicit ownership, exclusive student reservations, cancellation, privacy and deterministic recommendations.

Milestone commits: dfd1497 (schema/policy); 288a435 (engine/calendar); 79e12a9 (application/UI); 9a35df2 (fictional demo). Final presentation/documentation refinements are recorded in subsequent branch commits.